
Design Document for Synergy

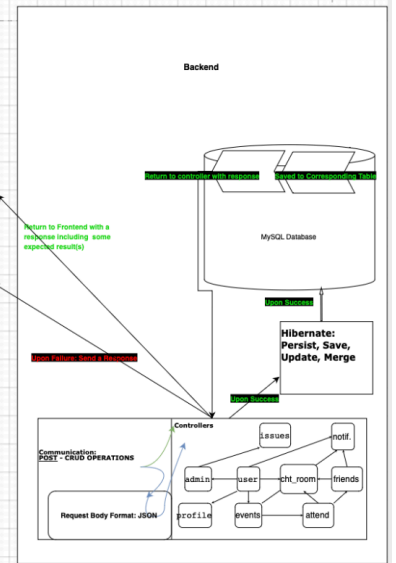
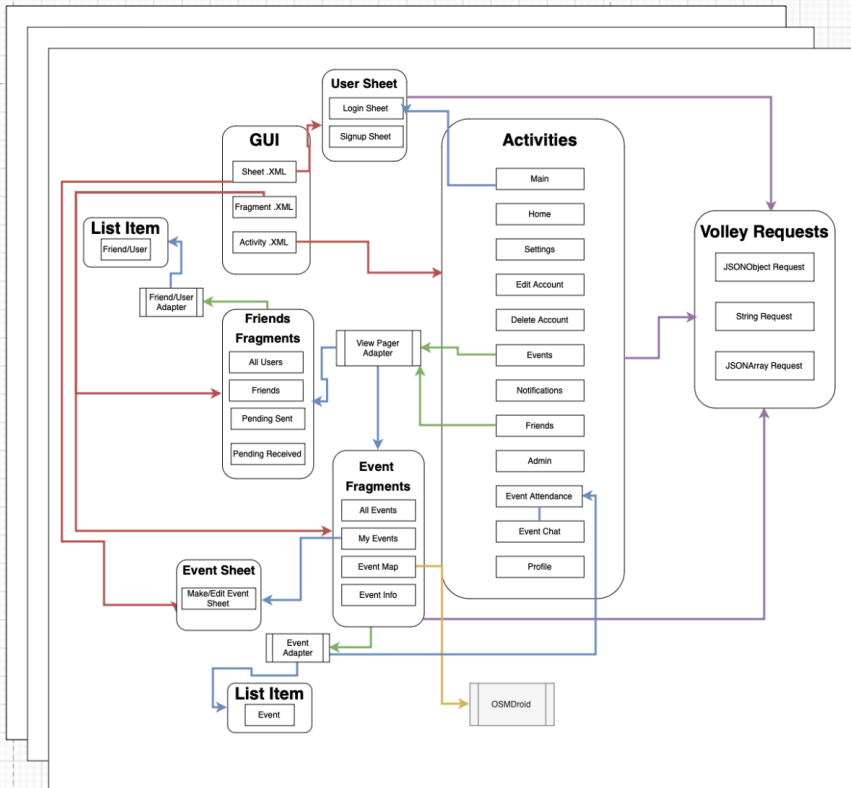
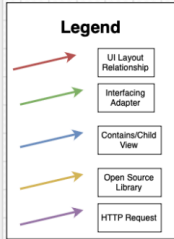
Group: 1_swarna_7

Ashwin Shrestha: 25% contribution

Morgan Crane: 25% contribution

Shranav Sakha: 25% contribution

Tanya Ken: 25% contribution



Block Diagram Description

FRONTEND (CURRENTLY IMPLEMENTED)

Friends Activity / Events Activity

- Uses a View Pager and a Tab Layout Element to switch between 4 views (3 views for Events). Uses a View Pager Adapter Class to interface with the 4 fragment views
 - All Users/Events Fragment: Lists all users/events, upon opening this view by making a GET Request.
 - Contains a RecyclerView that lists all the users / events: userList / eventList
 - Friends List / My Events Fragment: Lists the friends / events of the actor using a GET Request
 - Contains Recycler View that lists friends of the actor / lists events belonging to the actor
 - Sent Requests Fragment (Unique to Friends Activity): Lists users that the current actor has sent friend requests to.
 - Contains Recycler View that lists sent friend requests of the actor
 - Received Requests Fragment (Unique to Friends Activity): Lists users that have sent friend requests to the actor.
 - Contains Recycler View that lists received friend requests for the actor.
 - Map Node Fragment (Unique to Events Activity): Uses an Open Source Library OSMDroid to create map nodes to visualize where events are located similar to Google Maps.
 - Contains a OSMDroid Map View with custom markers set to display locations of events which we get through a GET Request.
 - Clicking on a Map Node opens a list of events that are happening in that location using another Recycler View and “Events” Item.
- For an item in each list of each fragment, there is a XML layout for each “Friend/ Event” Item:
 - Provides functionality and features using elements:

- Holder : Is the body of the “Friend / Events” item, in a Friends List Fragment, clicking on a Holder should open Friend Chat
- Unfriend / Add Friend button (Friends): Based on if it's an All Users Fragment or Friends List Fragment, this button will either be an Add Friend Button or an Unfriend Button, respectively. Using POST/DELETE Request
- Accept / Reject Button (Friends): If it's a Received Requests Fragment, these two buttons would appear and would provide functionality of either accepting or rejecting friend requests. Using POST Request
- Delete Button (Events): Deletes the Event button using the DELETE Request
- Attendance Button: Displays all users that are attending the event, using a GET Request.

Admin Activity

- Admin Dashboard is opened from the “Admin” button in Home.
 - OnCreate, it makes a GET request to fetch all users.
 - For each user returned, the activity also checks their current admin status:
 - isAdmin Tier 1
 - isAdmin Tier 2
 - else, they are Tier 0
 - Users are displayed in Recycler View using AdminUserAdapter.
 - Recycler View lists all users except the current super admin.
 - Each row shows
 - name
 - email
 - current Tier
 - Clicking a row opens AdminUserDetailActivity, passing:
 - userId
 - name
 - email
 - tier

Admin User Detail Activity

- When a user item in the AdminDashboard is clicked, AdminDashBoard passes selected user's id, name, email and current tier through an intent to AdminUserDetailActivity.
- AdminUserDetailActivity displays:
 - Name
 - Email
 - Current Tier ("Tier: 0 / 1 / 2")
- There are four buttons to manage privileges:
 - Grant Tier 1
 - Sends a POST request to promote a normal user (Tier 0) to a Tier 1 admin.
 - Upgrade to Tier 2
 - Sends a PUT request to upgrade from Tier 1
 - Sends a POST request to directly upgrade from tier 0.
 - Revoke Tier 2 (Keep Tier 1)
 - Sends a PUT request to demote Tier 2 to Tier 1 admin.
 - Remove Admin (To Tier 0)
 - Sends a DELETE request to remove all admin privileges so the user becomes a normal user (Tier 0).
 - After each successful request, the activity updates the local tier value and refreshes the UI state.
 - Buttons are enabled/disabled according to the user's current tier:
 - Tier 0: can be granted Tier 1 or Tier 2 admin status
 - Tier 1: can be upgraded to Tier 2 or removed as admin
 - Tier 2: can have their Tier 2 privileges revoked or removed as admin

BACKEND

Communication

Server:

- The backend server is built using Springboot with Apache Tomcat, which allows the application to handle RESTful communication between the frontend and the database.
- The backend is structured using 4 primary components: repository, model, service and controller.
 - Controller: Acts as the intermediary between the frontend and backend by defining the mapping URLs and endpoints that the frontend calls.
 - Service: Handles the logic and validation of each request before interacting with the database. Using the service helps clean up the controller methods.
 - Repository: The interface for database operations to access and modify stored data.
 - Model: Contains entity classes that define the structure and relationships of each table in the database. It includes the fields, constructors, getters and setters.
- A typical request from the frontend typically goes through the relevant controller, then the service method, and finally a repository call to update or retrieve data from the database.

Database:

- Uses a relational database that supports foreign keys, structured data, and relationships such as one-to-many, many-to-many etc...
- Synergy's database stores user account information, profile information, events hosted by users, event attendees, friendships between users, chatrooms and messages, notifications, map node information, and admin issues.
- Mappings:
 - GET: Request and retrieve information from the relevant database table.
 - POST: Send new information to the relevant database table.
 - PUT: Update relevant information on a database table.
 - DELETE: Check and send an identifier to delete relevant information from a database table.
- Tables: `admin`, `admin\_issue`, `admin\_issue\_event`, `admin\_issue\_user`, `associated\_events`, `chat\_members`, `chat\_messages`, `chat\_room`, `event\_attendees`, `event\_chat\_room`, `events`, `friendship`, `map\_nodes`, `notifications`, `priv\_chat\_room`, `profile`, `send\_message`, `user\_info`

- Maintains relationships between these tables, such as:
 - One user -> many events
 - Many users <-> many events (attendees)
 - One user -> many notifications
 - Many users <-> many chatrooms
 - One event <-> one chatroom (for group event messaging)

Controllers

- Each controller manages specific categories of API mappings and organizes backend communication.
 - User_Info Controller: Manages user account creation, login verification, editing user info, and retrieving user related data.
 - Profile Controller: Handles user profile creations, profile picture upload, personal details, and a one-to-one association with a user's account.
 - Events Controller: Stores and manages event information, handles attendees and event chatrooms.
 - Chatroom and Message Controllers: Linked to specific events and chatrooms while allowing users to join multiple chatrooms. Allows public and private chatroom creation.
 - Friendship Controller: Manages friendships and requests between users.
 - Admin Controllers: Handles event approval and issues with different tiers of privilege. (one-to-many relations to issues and events.)
 - Map_Nodes Controller: Stores location data for events that opt-in for nodes.

TABLE RELATIONSHIPS DIAGRAM

